In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler,LabelEncoder
import joblib
```

In [2]:
```
def reduce_mem_usage(df):
    start_mem = df.memory_usage().sum() / 1024**2
    print('Memory usage of dataframe is {:.2f} MB'.format(start_mem))

    for col in df.columns:
        col_type = str(df[col].dtype)

        if col_type == 'object' or col_type == 'category':
            df[col] = df[col].astype('category')
            continue

        try:
            c_min = df[col].min()
            c_max = df[col].max()
        except:
            df[col] = df[col].astype('category')
            continue

        if 'int' in col_type:
            if c_min > np.iinfo(np.int8).min and c_max < np.iinfo(np.int8).m
                df[col] = df[col].astype(np.int8)
            elif c_min > np.iinfo(np.int16).min and c_max < np.iinfo(np.int1
                df[col] = df[col].astype(np.int16)
            elif c_min > np.iinfo(np.int32).min and c_max < np.iinfo(np.int3
                df[col] = df[col].astype(np.int32)
            elif c_min > np.iinfo(np.int64).min and c_max < np.iinfo(np.int6
                df[col] = df[col].astype(np.int64)
        elif 'float' in col_type:
            if c_min > np.finfo(np.float16).min and c_max < np.finfo(np.floa
                df[col] = df[col].astype(np.float16)
            elif c_min > np.finfo(np.float32).min and c_max < np.finfo(np.fl
                df[col] = df[col].astype(np.float32)
            else:
                df[col] = df[col].astype(np.float64)
        else:
            df[col] = df[col].astype('category')

    end_mem = df.memory_usage().sum() / 1024**2
    print('Memory usage after optimization is: {:.2f} MB'.format(end_mem))
    print('Decreased by {:.1f}%'.format(100 * (start_mem - end_mem) / start_

    return df
```

In [3]:
```
df_id = pd.read_csv('/home/mostafashazly/Desktop/ieee-fraud-detection/ieee-f
df_id.head()
```

Out[3]:

	TransactionID	id-01	id-02	id-03	id-04	id-05	id-06	id-07	id-08	id-09	...	id-31	id 3
0	3663586	-45.0	280290.0	NaN	NaN	0.0	0.0	NaN	NaN	NaN	...	chrome 67.0 for android	NaN
1	3663588	0.0	3579.0	0.0	0.0	0.0	0.0	NaN	NaN	0.0	...	chrome 67.0 for android	24.
2	3663597	-5.0	185210.0	NaN	NaN	1.0	0.0	NaN	NaN	NaN	...	ie 11.0 for tablet	NaN
3	3663601	-45.0	252944.0	0.0	0.0	0.0	0.0	NaN	NaN	0.0	...	chrome 67.0 for android	NaN
4	3663602	-95.0	328680.0	NaN	NaN	7.0	-33.0	NaN	NaN	NaN	...	chrome 67.0 for android	NaN

5 rows × 41 columns

In [4]:
```
df_tran = pd.read_csv('/home/mostafashazly/Desktop/ieee-fraud-detection/ieee
df_tran.head()
```

Out[4]:

	TransactionID	TransactionDT	TransactionAmt	ProductCD	card1	card2	card3	c
0	3663549	18403224	31.95	W	10409	111.0	150.0	
1	3663550	18403263	49.00	W	4272	111.0	150.0	
2	3663551	18403310	171.00	W	4476	574.0	150.0	
3	3663552	18403310	284.95	W	10989	360.0	150.0	
4	3663553	18403317	67.95	W	18018	452.0	150.0	master

5 rows × 393 columns

In [5]:
```
df_trainid = reduce_mem_usage(df_id)
df_trans = reduce_mem_usage(df_tran)
```

```
Memory usage of dataframe is 55.34 MB
Memory usage after optimization is: 9.81 MB
Decreased by 82.3%
Memory usage of dataframe is 1533.96 MB
Memory usage after optimization is: 425.23 MB
Decreased by 72.3%
```

In [8]:
```
list_tittle = df_trainid[['TransactionID','DeviceType','DeviceInfo']]
final_data_for_test = pd.merge(df_trans,list_tittle,on="TransactionID",how="
```

In [9]:
```
final_data_for_test['TransactionAmt'] = final_data_for_test['TransactionAmt'
final_data_for_test['TransactionAmt'].nlargest(10)
```

Out[9]:
```
243076    10272.0
241064     9800.0
241816     9600.0
241075     9336.0
278061     9336.0
67842      9152.0
67847      9152.0
241823     8816.0
500634     7928.0
500741     7928.0
Name: TransactionAmt, dtype: float32
```

In [10]:
```
number_of_duplicats = final_data_for_test.duplicated().sum()
number_of_missing_values = final_data_for_test.isna().sum()

print(f"Number of duplicates in Test: {number_of_duplicats}")
print(f"Number of missing values in Test:\n{number_of_missing_values}")
```

```
Number of duplicates in Test: 0
Number of missing values in Test:
TransactionID          0
TransactionDT          0
TransactionAmt         0
ProductCD              0
card1                  0
                   ...
V337              430260
V338              430260
V339              430260
DeviceType        369760
DeviceInfo        391634
Length: 395, dtype: int64
```

In [11]:
```
final_data_for_test.columns.to_list()
```

Out[11]:
```
['TransactionID',
 'TransactionDT',
 'TransactionAmt',
 'ProductCD',
 'card1',
 'card2',
 'card3',
 'card4',
 'card5',
 'card6',
 'addr1',
 'addr2',
 'dist1',
 'dist2',
 'P_emaildomain',
 'R_emaildomain',
 'C1',
 'C2',
 'C3',
 'C4',
 'C5',
 'C6',
 'C7',
 'C8',
 'C9',
 'C10',
 'C11',
 'C12',
 'C13',
 'C14',
 'D1',
 'D2',
 'D3',
 'D4',
 'D5',
 'D6',
 'D7',
 'D8',
 'D9',
 'D10',
 'D11',
 'D12',
 'D13',
 'D14',
 'D15',
 'M1',
 'M2',
 'M3',
 'M4',
 'M5',
 'M6',
 'M7',
 'M8',
 'M9',
 'V1',
 'V2',
```

```
 'V339',
 'DeviceType',
 'DeviceInfo']
```

In [12]:
```
final_data_for_test['TransactionAmt_Log'] = np.log1p(final_data_for_test['Tr
```

```
/tmp/ipykernel_10119/1696773954.py:1: PerformanceWarning: DataFrame is highl
y fragmented.  This is usually the result of calling `frame.insert` many tim
es, which has poor performance.  Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fra
me.copy()`
  final_data_for_test['TransactionAmt_Log'] = np.log1p(final_data_for_test
['TransactionAmt'])
```

In [13]:
```
final_data_for_test['DeviceInfo'] = final_data_for_test['DeviceInfo'].cat.ac
final_data_for_test['DeviceType'] = final_data_for_test['DeviceType'].cat.ac


final_data_for_test['DeviceInfo'] = final_data_for_test['DeviceInfo'].fillna
final_data_for_test['DeviceType'] = final_data_for_test['DeviceType'].fillna
```

In [14]:
```
cat_cols_all = final_data_for_test.select_dtypes(include=['object', 'categor
num_cols_all = final_data_for_test.select_dtypes(include=['number', 'float16
```

In [15]:
```
for col in cat_cols_all:
    if 'Unknown' not in final_data_for_test[col].cat.categories:
        final_data_for_test[col] = final_data_for_test[col].cat.add_categori
    final_data_for_test[col] = final_data_for_test[col].fillna('Unknown')
final_data_for_test[num_cols_all] = final_data_for_test[num_cols_all].fillna
print("Total Missing in Full Data:", final_data_for_test.isna().sum().sum())
```

```
Total Missing in Full Data: 0
```

In [16]:
```
final_data_for_test['Hour'] = (final_data_for_test['TransactionDT'] / (3600)
final_data_for_test['Hour'].head(10)
```

```
/tmp/ipykernel_10119/541149381.py:1: PerformanceWarning: DataFrame is highly
fragmented.  This is usually the result of calling `frame.insert` many time
s, which has poor performance.  Consider joining all columns at once using p
d.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fram
e.copy()`
  final_data_for_test['Hour'] = (final_data_for_test['TransactionDT'] / (360
0)) % 24
```

Out[16]:
```
0    0.006667
1    0.017500
2    0.030556
3    0.030556
4    0.032500
5    0.034167
6    0.041667
7    0.051944
8    0.056944
9    0.060000
Name: Hour, dtype: float64
```

In [17]:
```
final_data_for_test['DeviceInfo'] = final_data_for_test['DeviceInfo'].str.lc
```

In [18]:
```
final_data_for_test['device_combined'] = final_data_for_test['DeviceType'].a
print(final_data_for_test['device_combined'].value_counts().head(10))
```

```
device_combined
Unknown_unknown                 369742
desktop_windows                  44984
mobile_ios device                18720
mobile_unknown                   11268
desktop_macos                    11149
desktop_unknown                  10624
desktop_trident/7.0               4879
desktop_rv:11.0                    735
mobile_sm-g532m build/mmb29t       664
desktop_rv:63.0                    420
Name: count, dtype: int64
```

```
/tmp/ipykernel_10119/3931256796.py:1: PerformanceWarning: DataFrame is highl
y fragmented.  This is usually the result of calling `frame.insert` many tim
es, which has poor performance.  Consider joining all columns at once using
pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fra
me.copy()`
  final_data_for_test['device_combined'] = final_data_for_test['DeviceTyp
e'].astype(str) + "_" + final_data_for_test['DeviceInfo'].astype(str)
```

In [19]:
```
final_data_for_test = final_data_for_test.copy()
```

In [21]:
```
addr1_counts_train = joblib.load('addr1_counts.pkl')

final_data_for_test['addr1_freq'] = final_data_for_test['addr1'].astype('flc

final_data_for_test['addr1_freq'] = final_data_for_test['addr1_freq'].fillna

print(final_data_for_test[['addr1', 'addr1_freq']].head())
```

```
   addr1  addr1_freq
0  170.0      2001.0
1  299.0     46335.0
2  472.0      8478.0
3  205.0      5725.0
4  264.0     39870.0
```

```
/home/mostafashazly/miniconda3/lib/python3.13/site-packages/pandas/io/format
s/format.py:1466: RuntimeWarning: overflow encountered in cast
  has_large_values = (abs_vals > 1e6).any()
```

In [22]:
```
final_data_for_test['email_check'] = (final_data_for_test['P_emaildomain'].a

print(final_data_for_test['email_check'].value_counts())
```

```
email_check
0    345317
1    161374
Name: count, dtype: int64
```

In [23]:
```
final_data_for_test['dist1_is_unknown'] = (final_data_for_test['dist1'] == -
final_data_for_test['dist1_log'] = np.log1p(final_data_for_test['dist1'].rep
```

In [25]:
```
scaler_d_train = joblib.load('scaler_d.pkl')
d_nums = [1, 2, 3, 4, 5, 10, 11, 15]
d_original_cols = [f'D{i}' for i in d_nums]

d_log_cols = [f'D{i}_log' for i in d_nums]

for col in d_original_cols:
    final_data_for_test[f'{col}_log'] = np.log1p(final_data_for_test[col].cl

final_data_for_test[d_log_cols] = scaler_d_train.transform(final_data_for_te

if 'D1' in final_data_for_test.columns and 'D2' in final_data_for_test.colum
    final_data_for_test['D1_minus_D2'] = final_data_for_test['D1'] - final_c
```

In [26]:
```
scaler_c_train = joblib.load('scaler_c.pkl')

c_cols = [f'C{i}' for i in range(1, 15) if f'C{i}' in final_data_for_test.cc
c_log_cols = [f'{col}_log' for col in c_cols]

for col in c_cols:
    final_data_for_test[f'{col}_is_unknown'] = (final_data_for_test[col] ==
    final_data_for_test[f'{col}_log'] = np.log1p(final_data_for_test[col].cl

final_data_for_test[c_log_cols] = scaler_c_train.transform(final_data_for_te
```

In [27]:
```
m_cols = [f'M{i}' for i in range(1, 10) if f'M{i}' in final_data_for_test.cc

for col in m_cols:

    final_data_for_test[col] = final_data_for_test[col].astype(str).str.lowe


    final_data_for_test[col] = final_data_for_test[col].map({
        't': 1, 'f': 0,
        'true': 1, 'false': 0
    }).fillna(-1).astype('int8')

print(f"Sample of M columns in Test after encoding:")
print(final_data_for_test[m_cols].head())
```

```
Sample of M columns in Test after encoding:
   M1  M2  M3  M4  M5  M6  M7  M8  M9
0   1   1   0  -1  -1   0   1   1   1
1   1   0   0  -1  -1   0  -1  -1  -1
2   1   1   0  -1   0   0   0   0   0
3   1   1   1  -1  -1   1  -1  -1  -1
4   1   1   1  -1  -1   0   0   1   1
```

In [29]:
```
scaler_v_train = joblib.load('scaler_v.pkl')


v_cols_train = scaler_v_train.feature_names_in_
```

```
for col in v_cols_train:
    if col not in final_data_for_test.columns:
        final_data_for_test[col] = -1

final_data_for_test[v_cols_train] = scaler_v_train.transform(final_data_for_

print(f"Done! Scaled {len(v_cols_train)} V-columns using Train feature names
```

```
Done! Scaled 292 V-columns using Train feature names.
```

In [30]:
```
final_data_for_test['hour'] = (final_data_for_test['TransactionDT'] // 3600)
final_data_for_test['day_of_week'] = (final_data_for_test['TransactionDT'] /
```

```
/tmp/ipykernel_10119/267950398.py:1: PerformanceWarning: DataFrame is highly
fragmented.  This is usually the result of calling `frame.insert` many time
s, which has poor performance.  Consider joining all columns at once using p
d.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fram
e.copy()`
  final_data_for_test['hour'] = (final_data_for_test['TransactionDT'] // 360
0) % 24
/tmp/ipykernel_10119/267950398.py:2: PerformanceWarning: DataFrame is highly
fragmented.  This is usually the result of calling `frame.insert` many time
s, which has poor performance.  Consider joining all columns at once using p
d.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = fram
e.copy()`
  final_data_for_test['day_of_week'] = (final_data_for_test['TransactionDT']
// (3600 * 24)) % 7
```

In [31]:
```
cols_to_drop = [
    'TransactionID', 'TransactionDT', 'TransactionAmt',
    'dist1', 'Hour', 'D1', 'D2', 'D3', 'D4', 'D5', 'D10', 'D11', 'D15']

c_original = [f'C{i}' for i in range(1, 15)]
cols_to_drop.extend(c_original)

df_final = final_data_for_test.drop(columns=cols_to_drop)

print(f"Original columns: {len(final_data_for_test.columns)}")
print(f"Final columns: {len(df_final.columns)}")
```

```
Original columns: 441
Final columns: 414
```

In [32]:
```
df_final.columns.to_list()
```

Out[32]:
```
['ProductCD',
 'card1',
 'card2',
 'card3',
 'card4',
 'card5',
 'card6',
 'addr1',
 'addr2',
 'dist2',
 'P_emaildomain',
 'R_emaildomain',
 'D6',
 'D7',
 'D8',
 'D9',
 'D12',
 'D13',
 'D14',
 'M1',
 'M2',
 'M3',
 'M4',
 'M5',
 'M6',
 'M7',
 'M8',
 'M9',
 'V1',
 'V2',
 'V3',
 'V4',
 'V5',
 'V6',
 'V7',
 'V8',
 'V9',
 'V10',
 'V11',
 'V12',
 'V13',
 'V14',
 'V15',
 'V16',
 'V17',
 'V18',
 'V19',
 'V20',
 'V21',
 'V22',
 'V23',
 'V24',
 'V25',
 'V26',
 'V27',
 'V28',
```

```
 'C5_is_unknown',
 'C5_log',
 'C6_is_unknown',
 'C6_log',
 'C7_is_unknown',
 'C7_log',
 'C8_is_unknown',
 'C8_log',
 'C9_is_unknown',
 'C9_log',
 'C10_is_unknown',
 'C10_log',
 'C11_is_unknown',
 'C11_log',
 'C12_is_unknown',
 'C12_log',
 'C13_is_unknown',
 'C13_log',
 'C14_is_unknown',
 'C14_log',
 'hour',
 'day_of_week']
```

In [33]:
```
base_cols = ['ProductCD', 'card1', 'card2', 'card3', 'card4', 'card5', 'carc
             'P_emaildomain', 'R_emaildomain']

m_cols = [f'M{i}' for i in range(1, 10)]
v_cols = [f'V{i}' for i in range(1, 322)]


d_nums = [1, 2, 3, 4, 5, 10, 11, 15]
d_features = [f'D{i}_log' for i in d_nums] + ['D1_is_unknown', 'D1_minus_D2'

c_nums = range(1, 15)
c_features = [f'C{i}_log' for i in c_nums] + [f'C{i}_is_unknown' for i in c_

other_features = [
    'DeviceType', 'DeviceInfo', 'TransactionAmt_Log', 'Hour_int',
    'device_combined', 'addr1_freq', 'email_check',
    'dist1_is_unknown', 'dist1_log', 'hour', 'day_of_week'
]

full_train_cols = base_cols + m_cols + v_cols + d_features + c_features + ot


X_test = final_data_for_test.reindex(columns=full_train_cols)

X_test = X_test.fillna(-1)

print(f"Done! Test data shape: {X_test.shape}")
```

```
Done! Test data shape: (506691, 390)
```

In [34]:
```
dict_encoders = joblib.load('label_encoders_dict.pkl')
scaler_final = joblib.load('min_max_scaler.pkl')
```

```
X_test = final_data_for_test.reindex(columns=scaler_final.feature_names_in_)

print("Encoding categorical columns...")
for col, le in dict_encoders.items():
    if col in X_test.columns:

        X_test[col] = X_test[col].astype(str).map(lambda s: s if s in le.cla
        X_test[col] = le.transform(X_test[col])

X_test = X_test.fillna(-1)

print("Scaling all features...")
X_test_scaled = pd.DataFrame(
    scaler_final.transform(X_test),
    columns=X_test.columns
)

X_test_scaled.to_csv('test_final_processed.csv', index=False)

print(f"Final Test Shape: {X_test_scaled.shape}")
```

```
Encoding categorical columns...
Scaling all features...
Final Test Shape: (506691, 361)
```

In [ ]:

```
print(f"Train columns count: {len(scaler_final.feature_names_in_)}")
print(f"Test columns count: {X_test_scaled.shape[1]}")

diff = set(scaler_final.feature_names_in_) - set(X_test_scaled.columns)
print(f"Columns in Train but not in Test: {diff}")
```

```
Train columns count: 361
Test columns count: 361
Columns in Train but not in Test: set()
```